HTML

- Used to create the **structure** of a webpage.
- Uses tags like `<h1>`, `<p>`, `<div>`, `<form>`.
- Defines text, images, links, tables, and forms.

CSS

- Used to **style** HTML pages.
- Controls colors, fonts, spacing, and layout.
- Uses selectors, properties, Flexbox, and Grid.

JavaScript

- Used to add **functionality and interactivity**.
- Handles events, buttons, forms, and dynamic content.
- Can communicate with APIs and backends.

SQL

- Used to **manage databases**.
- Performs operations like `SELECT`, `INSERT`, `UPDATE`, and `DELETE`.
- Works with tables, rows, columns, and relationships.

Adding JavaScript to a Web Page

There are **3 ways** to add JavaScript:

1. Inline JavaScript

- JavaScript is written directly inside an HTML element.

HTML

```html
<button onclick="alert('Hello')">Click Me</button>
```

2. Internal JavaScript

- JavaScript is written inside the `<script>` tag.

HTML

```html
<script>
    alert("Hello World");
</script>
```

3. External JavaScript

- JavaScript is written in a separate `.js` file.
- The file is connected using `<script src="">`.

HTML:

```HTML
<script src="script.js"></script>
```

script.js:

```JavaScript
alert("Hello World");
```

External JavaScript is generally preferred because it keeps HTML and JavaScript separate and makes the code easier to maintain.

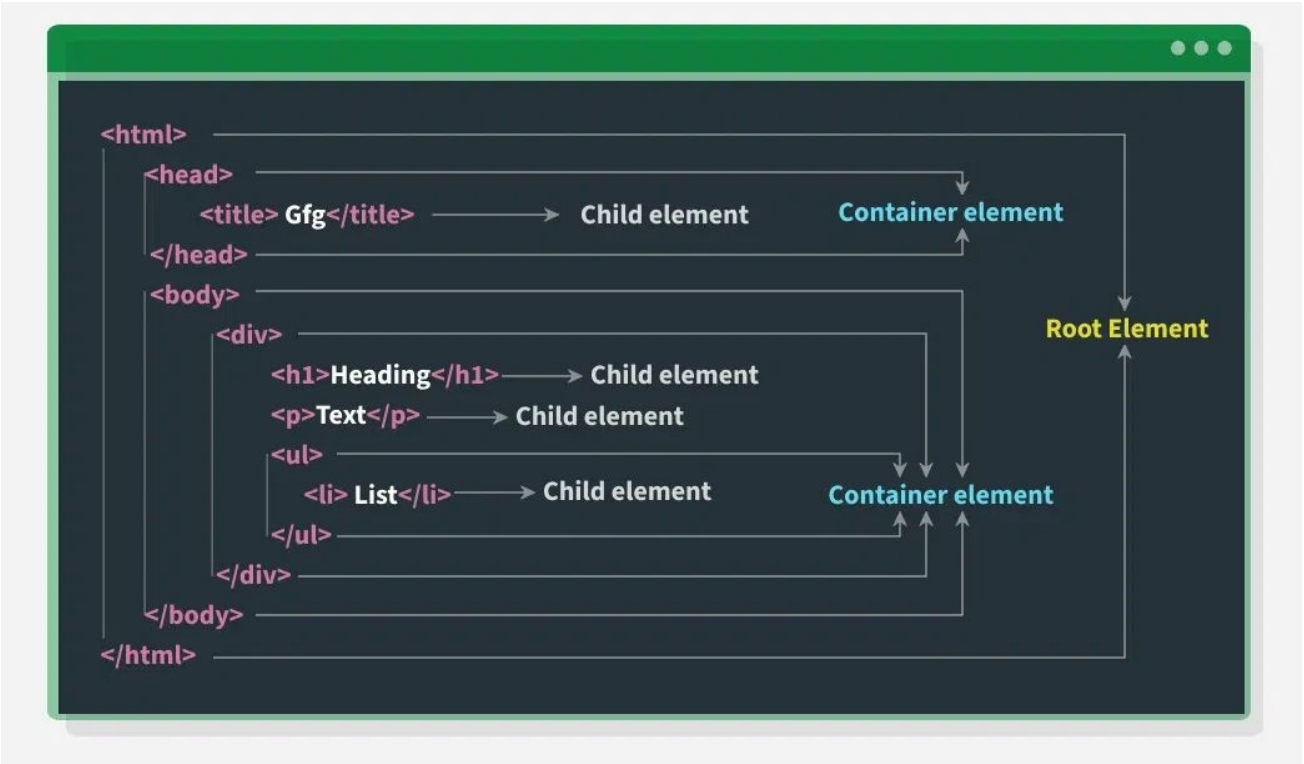
`<script>` Tag

- The `<script>` tag is used to **add JavaScript to an HTML page**.
- It can contain JavaScript code directly or link to an external `.js` file.

```
<script>
    alert("Hello World");
</script>
```

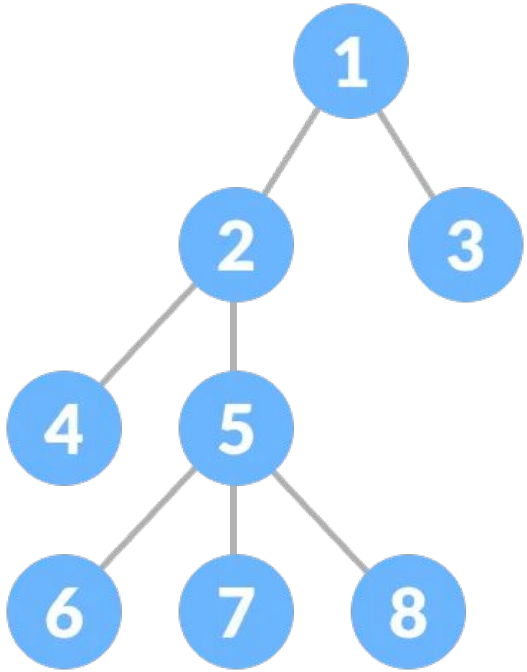
DOM (Document Object Model)

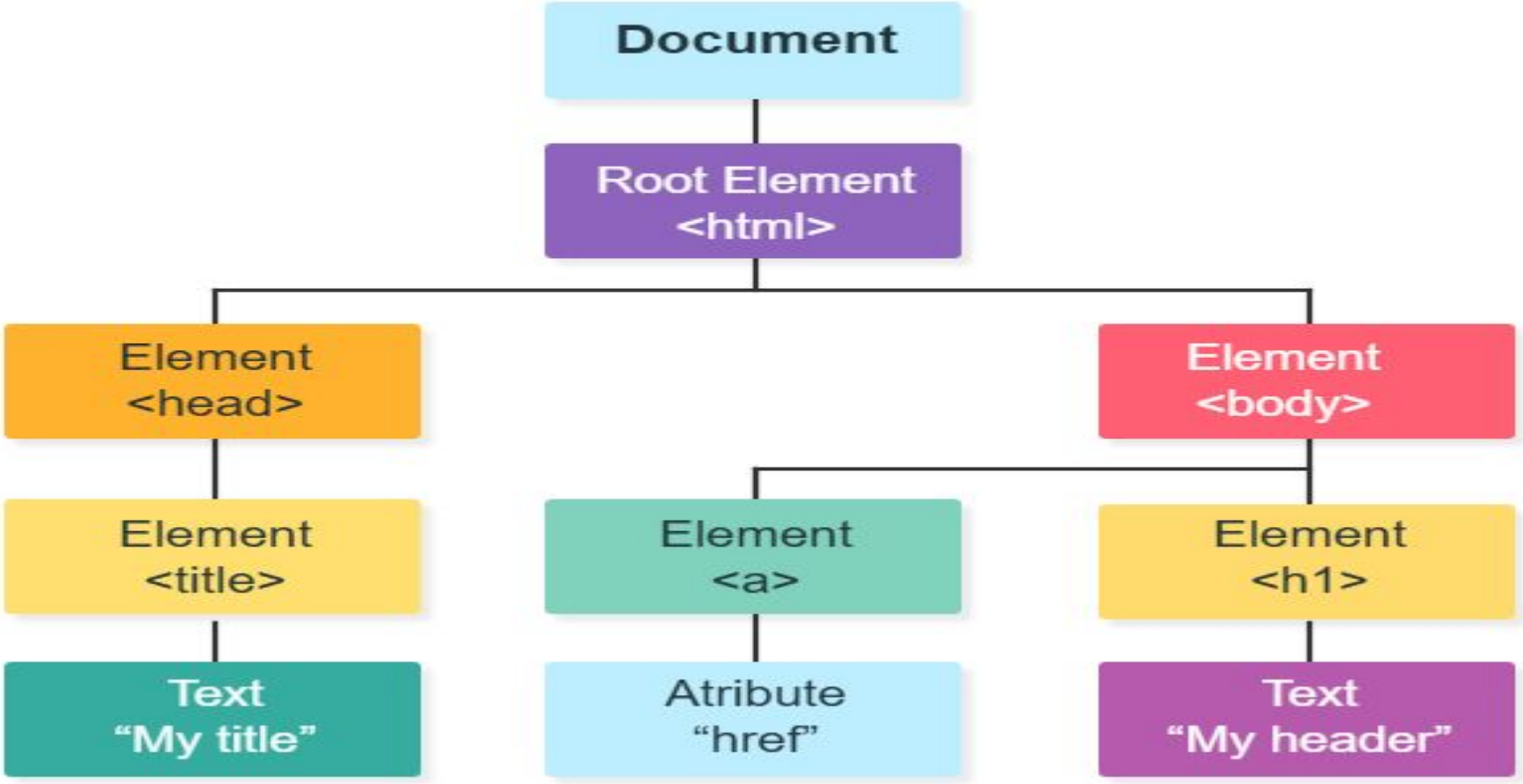
- DOM is a **tree-like representation of an HTML webpage**.
- JavaScript uses the DOM to **access and change HTML elements**.
- Every HTML element becomes an **object/node** in the DOM.



Tree Data Structure

- A **tree** is a **non-linear data structure** used to represent hierarchical data.
- It consists of **nodes** connected by edges.
- The top node is called the **root**.
- A node can have **child nodes**.
- A node above another node is called its **parent**.
- A node with no children is called a **leaf node**.
- Examples include **file systems, organization charts, and HTML DOM**.

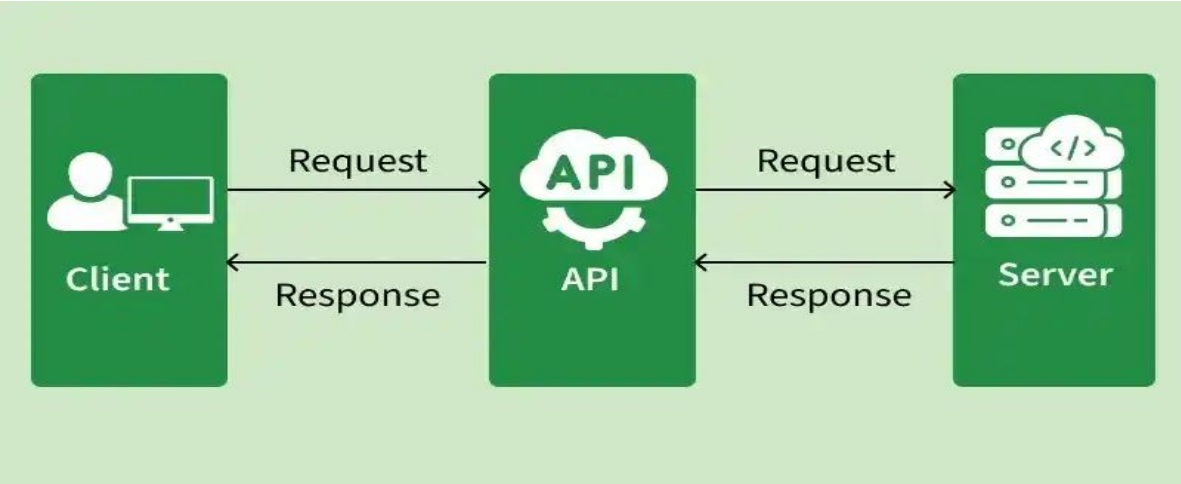



Document
Root Element
<html>
Element
<head>
Element
<body>
Element
<title>
Element
<a>
Element
<h1>
Text
“My title”
Atribute
“href”
Text
“My header”

API

API = Application Programming Interface

- An API is a **way for two software programs to communicate with each other**.
- It provides predefined **methods or rules** for requesting and exchanging data.
- For example, JavaScript can use an API to get data from a server.



DOM API

The `document` variable stores a (very large) object. That object represents the DOM, and has methods that you can call to access and manipulate the DOM. It also has properties about the DOM.

- **DOM API** is a set of **methods and properties provided by the browser** to work with HTML elements.
- JavaScript uses the DOM API to **find, change, add, or remove elements** on a webpage.

DOM ACCESS METHODS

DOM access methods are used by JavaScript to find HTML elements on a webpage.

`console.log()` in JavaScript

Definition: `console.log()` is a JavaScript method used to display messages, variables, values, and results in the browser's developer console.

Syntax

JavaScript

```
console.log(value);
```

Basic Printing Functions in JavaScript

JavaScript provides different methods to display output on the webpage or in the browser console.

Function	Description	Example
`console.log()`	Prints output in the browser console.	`console.log("Hello");`
`console.warn()`	Displays a warning message in the console.	`console.warn("Warning!");`
`console.error()`	Displays an error message in the console.	`console.error("Error!");`
`console.info()`	Displays an informational message in the console.	`console.info("Information");`
`alert()`	Displays a popup message.	`alert("Hello!");`
`document.write()`	Writes content directly into the HTML document.	`document.write("Hello");`
`document.writeln()`	Writes content followed by a newline character.	`document.writeln("Hello");`
`innerHTML`	Displays or inserts HTML content inside an element.	`document.body.innerHTML = "Hello";`
`innerText`	Displays or changes the visible text of an element.	`document.body.innerText = "Hello";`

`document.body.innerHTML` is a JavaScript property used to access or change the HTML content inside the `<body>` tag of a webpage.

- `document` represents the HTML document.
- `body` refers to the `<body>` element of the webpage.
- `innerHTML` represents the HTML content inside that element.

```
<body>
    <h1>Hello World</h1>

    <script>
        document.body.innerHTML = "Welcome to JavaScript!";
    </script>
</body>
```



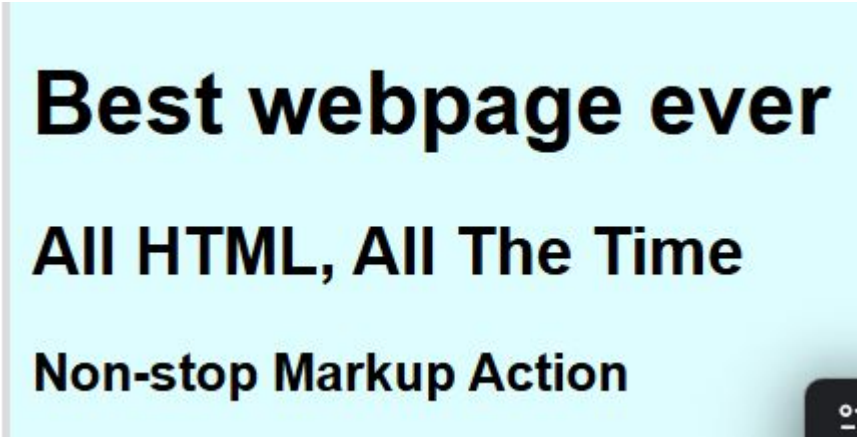
```
<!DOCTYPE html>
<html>
   <head>
      <meta charset="utf-8">
      <title>Putting JS in a webpage</title>
      <style>
         body {
            font-family: Arial, sans-serif;
            background: rgb(222, 253, 255);
         }
      </style>

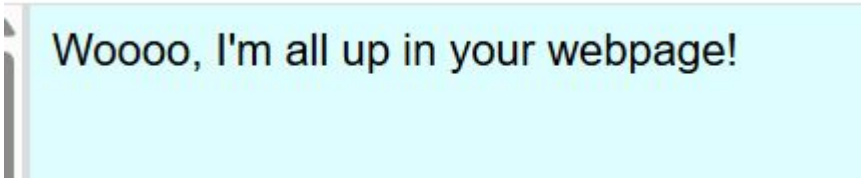
   </head>
   <body>
      <h1>Best webpage ever</h1>

      <h2>All HTML, All The Time</h2>

      <h3>Non-stop Markup Action</h3>

      <script>
         var four = 2 + 2;
         console.log("Wheeee!"); // cmd-opt-I, ctrl-option-I
         document.body.innerHTML = "Woooo, I'm all up in your webpage!";
      </script>
   </body>
</html>
```





DOM METHODS

The DOM (Document Object Model) allows JavaScript to access, select, and modify HTML elements on a webpage.

1. `document.getElementById()`

This method selects an HTML element using its unique `id`.

```
<p id="demo">Hello World</p>

<script>
    // Select the element using its ID.
    let x = document.getElementById("demo");

    // Print the element's content.
    console.log(x.innerHTML);
</script>
```

2. `document.getElementsByTagName()`

This method selects all HTML elements with a specified tag name, such as `p`, `h1`, or `div`.

```
<p>Hello</p>
<p>Welcome</p>

<script>
    // Select all paragraph elements.
    let x = document.getElementsByTagName("p");

    // Print the content of the first paragraph.
    console.log(x[0].innerHTML);

    // Print the content of the second paragraph.
    console.log(x[1].innerHTML);
</script>
```

3. `document.getElementsByClassName()`

This method selects all HTML elements that have a specified class name.

```
<p class="info">First paragraph</p>
<p class="info">Second paragraph</p>

<script>
    // Select all elements with class "info".
    let x = document.getElementsByClassName("info");

    // Print the first matching element's content.
    console.log(x[0].innerHTML);
</script>
```

4. `document.querySelector()`

This method selects the **first element** that matches a specified CSS selector.

Example:

HTML

```html
<p class="info">Hello</p>
<p class="info">Welcome</p>

<script>
    // Select the first element with class "info".
    let x = document.querySelector(".info");

    console.log(x.innerHTML);
</script>
```

5. `document.querySelectorAll()`

This method selects **all elements** that match a specified CSS selector.

Example:

HTML

```html
<p>Hello</p>
<p>Welcome</p>

<script>
    // Select all paragraph elements.
    let x = document.querySelectorAll("p");

    console.log(x[0].innerHTML);
    console.log(x[1].innerHTML);
</script>
```

6. `document.getElementsByName()`

This method selects elements based on their `name` attribute. It is commonly used with form elements.

Example:

HTML

```html
<input type="text" name="username" value="Ved">

<script>
    // Select elements having name="username".
    let x = document.getElementsByName("username");

    console.log(x[0].value);
</script>
```